

Graphbase

Knowledge Graph + RAG Studio: a complete walkthrough of the code

Repository: github.com/monish78/KnowledgeFabric | Written 30 Sep 2026

How to use this guide. Chapters 1-3 give the big picture (what the product does, how the containers fit together, where every file lives). Chapters 4-18 walk through each part of the code in the order a request or a file flows through it, naming the file and function that does each step. Chapter 19 lists every API endpoint with the role it requires, chapter 20 covers the React screens, chapters 21-23 cover demo data, the test dataset and the tests (with the latest results), chapter 24 covers deployment, and chapter 25 is a list of questions people are likely to ask, with answers.

File references look like `backend/app/loader.py: plan_load()`. Names in `Courier` are real identifiers you can search for in the code.

Contents

1. What the application does	3
2. Architecture	4
3. Repository map	5
4. Configuration (.env)	6
5. The Postgres database	7
6. Authentication and sessions	8
7. Access control	9
8. Background jobs and pipelines	10
9. Reading spreadsheets (tabular.py)	11
10. The graph schema and Cypher generation	12
11. The LLM factory (llm.py)	13
12. Knowledge-graph extraction (extraction.py)	14
13. Review and Submit	16
14. Loading rows into Neo4j (loader.py)	17
15. Neo4j modes and query safety (graphstore.py)	18
16. Adding data (pipeline 7)	19
17. RAG documents (rag.py)	20
18. Chat (chat.py and cypher_repair.py)	21
19. API reference	22
20. The React frontend	23
21. Admin commands and demo data	24
22. The test dataset	25
23. Tests and results	26
24. Deployment and operations	28
25. Questions you may be asked	29

1. What the application does

Graphbase lets people turn company data into something they can ask questions of, and control who else can.

- **Knowledge graphs from spreadsheets.** A user uploads a CSV or XLSX file with a name, domain and sub-domain. In the background an LLM works out what the rows describe (suppliers, products, orders...), how they are related, and which columns hold personal data (PII). The user reviews and edits that proposal on the Review screen. Only when they press Submit is the graph written to Neo4j.
- **RAG stores from documents.** A user uploads PDF, DOCX or TXT files. They are split into chunks, embedded and stored in ChromaDB automatically, with no review step. Documents are PII-scanned too.
- **Ownership and sharing.** Whoever creates a knowledge base (KB) is its owner. Only the owner can grant or revoke access. Every grant and revoke is written to an audit table (kb_access).
- **Adding data later.** New spreadsheet rows are loaded using the schema that was already approved (no review screen); rows that don't fit are rejected and listed in a report. New documents are added to the same vector store.
- **Chat.** For a graph KB, the LLM writes a Cypher query, the app checks and runs it, and the LLM answers from the results; the UI shows the Cypher and the graph path. For a RAG KB, the most relevant passages are retrieved and the LLM answers citing them. Access is checked on the server for every question.

Roles

Role	Can do
owner	Everything on that KB: review/submit the extracted schema, grant and revoke access, add data, chat, see the access log, re-run extraction.
user	Chat and add data (decision taken with you on 29 Sep). Cannot review, grant, revoke or see the access page.
anyone else	Nothing. The KB does not appear in their list and every endpoint returns 403.

Decisions recorded in CLAUDE.md

- Ollama is the active LLM (local test: qwen2.5:3b; work system: qwen2.5:7b-instruct); embeddings `nomic-embed-text`. Azure OpenAI GPT-4.1 code is written but commented out.
- Sign-in: user ID + password for local testing, Keycloak in production. Authentication and sessions are handled entirely by the backend (server-side sessions, HttpOnly cookie, no tokens in the browser).
- Neo4j: Community (one database) locally, Enterprise (one database per KB) supported by `NEO4J_MODE`.
- Audit columns (`created_at`, `updated_at`, `modified_by`) on every table, and a `kb_pii_fields` table filled automatically - both requested by your manager.
- KB names are unique across all users. RAG and graph KBs are shown in one list. No sign-up screen (CLI creates users).

2. Architecture

Everything runs with Docker Compose (`docker-compose.yml`). The browser only talks to the frontend container; the Vite dev server forwards every `/api/...` request to the backend.

```
Browser --> frontend (React/Vite, :5173) --/api--> backend (FastAPI, :8000)
                                     |-- Postgres (users, KBs, access, audit, jobs, PII) :5433
                                     |-- Neo4j (knowledge graphs) :7474/
                                     |-- ChromaDB (document chunks + vectors) :8001
                                     |-- Ollama (LLM + embeddings, on the host) :11434
                                     `-- Keycloak (production sign-in; optional local copy) :8080
```

Service	Image / build	Purpose and notes
postgres	postgres:16-alpine	Relational data. Host port 5433 because 5432 was taken on the laptop. Volume pgdata.
neo4j	neo4j:5-community (NEO4J_IMAGE)	Graphs. Heap/pagecache sizes from .env. Health check runs cypher-shell.
chroma	chromadb/chroma:0.6.3	Vector store (client pinned to the same version: chromadb-client 0.6.3).
backend	backend/Dockerfile (python:3.11-slim)	FastAPI app, background jobs, CLI and tests. Mounts ./backend/app, ./backend/tests, ./data (read by the dataset tests and demo seeding) and an uploads volume.
frontend	frontend/Dockerfile (node:20-alpine)	Vite dev server with the React app; proxies /api to backend:8000.
ollama-proxy (profile)	alpine/socat	Only for Linux hosts whose Ollama listens on 127.0.0.1: forwards 172.17.0.1:11435 to it, so containers can reach it without exposing Ollama to the network.
ollama (profile)	ollama/ollama	Optional: run Ollama inside Compose instead of on the host.
keycloak (profile)	quay.io/keycloak/keycloak:26.0	Optional local Keycloak (dev mode) with realm file keycloak/graphbase-realm.json and the six test users. Not used on the work system.

Optional services are switched on with `COMPOSE_PROFILES` in `.env` (e.g. `ollama-proxy`, `keycloak`).

What happens on backend start-up (backend/app/main.py)

- `run_migrations()` applies any new SQL file in `backend/app/migrations` (once each, guarded by a Postgres advisory lock).
- `open_pool()` opens the Postgres connection pool (`psycopg 3`, dict rows, 1-10 connections).
- `jobs.recover_interrupted()` marks jobs that were running when the server stopped as failed, so nothing is stuck at 'Extracting' forever.
- Routers: `app/api/auth.py` and `app/api/kbs.py`. GET `/api/health` checks Postgres, Neo4j, Chroma and Ollama (models present).

Libraries used

Area	Library (pinned in backend/requirements.txt)
Web API	fastapi 0.115, uvicorn, pydantic-settings, python-multipart (uploads)
Databases	psycopg 3 + psycopg_pool (Postgres), neo4j 5.27 driver, chromadb-client 0.6.3
LLM	langchain-core, langchain-ollama (ChatOllama, OllamaEmbeddings), langgraph (chat flow), langchain-text-splitters (RAG chunking); langchain-openai commented out for Azure
Auth	PyJWT[crypto] (verifies Keycloak ID tokens), cryptography (encrypts stored Keycloak tokens), bcrypt, httpx
Files	openpyxl (XLSX), csv (stdlib), pypdf (PDF), python-docx (DOCX)
Frontend	react 18, react-router-dom 6, vite 5 (no UI framework; styles in src/styles.css)

3. Repository map

Path	What it is
CLAUDE.md	Spec + every decision taken with you. Read this first when changing the code.
README.md	How to run, test, seed demo data and switch to production.
docker-compose.yml, .env.example	The stack and every setting (copy .env.example to .env).
docs/	Mockups (PNG/PDF/HTML), TEST_RESULTS.md, this guide.
keycloak/graphbase-realm.json	Local test realm: client graphbase-app + 6 users (password test1234).
data/generate_dataset.py	Builds the deliberately messy test dataset + expected answers (deterministic).
data/samples/	Generated files: supplier_orders.xlsx, supplier_orders_october.xlsx, warehouse_update.csv, finance_ledger.csv, returns_policy.pdf, vendor_handbook.docx, warehouse_sop.txt, edge_cases/.
data/expected/manifest.json	Ground truth: counts, rejected rows, PII columns, question/answer pairs.
backend/app/config.py	Settings class (reads environment variables).
backend/app/db.py	Postgres pool + migration runner.
backend/app/migrations/	001_init.sql: all business tables and the updated_at trigger; 002_sessions.sql: sessions and pending Keycloak sign-ins.
backend/app/auth.py, api/auth.py	Server-side sessions, local sign-in, the Keycloak OIDC flow; /api/auth routes.
backend/app/rules.py	Patterns shared by PII detection (e-mail, phone, ID numbers, organisation words).
backend/app/kb.py	KB catalog and access control (require_access, grant, revoke, access log).
backend/app/jobs.py	Background jobs with step progress, cancel, restart recovery.
backend/app/pipelines.py	The work each job does: extract, build, add-data, RAG ingest, PII rows.
backend/app/tabular.py	Reads CSV/XLSX, finds headers, parses messy values, profiles columns.
backend/app/extraction.py	LLM schema extraction + evidence checks + PII detection for spreadsheets.
backend/app/graph_schema.py	Schema format, name normalisation, validation, Cypher generation.
backend/app/loader.py	Validates rows and writes nodes/relationships (initial build and add-data).
backend/app/graphstore.py	Neo4j access per KB (single/multi mode), read-only checks, KB scoping of Cypher.
backend/app/rag.py	Document parsing, chunking, Chroma storage/retrieval, document PII scan.
backend/app/llm.py	get_llm() / get_embeddings() factory + JSON helpers.
backend/app/chat.py	LangGraph chat flow for graphs; RAG answers.
backend/app/cypher_repair.py	Fixes and lints LLM-written Cypher against the schema.
backend/app/api/kbs.py	All KB, job, review, access, add-data, runs, PII and chat endpoints.
backend/app/cli.py, demo.py	Admin commands (users, demo data) and the demo knowledge bases.
backend/tests/	173 fast tests + real-LLM benchmark (test_llm_quality.py).
e2e/test_ui.py	Playwright browser tests of every screen.
frontend/src/	React app: App.jsx (routes), auth.js, api.js, components/common.jsx, pages/*.jsx, styles.css.
backend/ruff.toml, frontend/eslint.config.js	Lint settings (Python: ruff; React: ESLint). Both report no issues.

4. Configuration (.env)

backend/app/config.py defines Settings (pydantic-settings): every field maps to an environment variable of the same name in upper case. get_settings() caches one instance. Compose passes .env to the backend (env_file) and overrides the in-network hosts (postgres, neo4j, chroma).

Variable	Meaning
POSTGRES_DB/USER/PASSWORD, POSTGRES_HOST_PORT	Postgres credentials; host port for tools (5433).
NEO4J_USER/PASSWORD	Neo4j login. Neo4j 5 refuses 'neo4j' as the password and needs 8+ characters.
NEO4J_MODE	single (Community: all KBs in one DB, separated by KB labels) or multi (Enterprise: one DB per KB).
NEO4J_IMAGE, NEO4J_ACCEPT_LICENSE, NEO4J_HEAP, NEO4J_PAGECACHE	Neo4j edition, licence acceptance, memory.
CHROMA_VERSION, CHROMA_HOST_PORT	Chroma server version (keep equal to the client) and host port.
LLM_PROVIDER	ollama (active) or azure (after uncommenting the Azure code).
OLLAMA_BASE_URL, OLLAMA_CHAT_MODEL, OLLAMA_EMBED_MODEL	Where Ollama is and which models to use.
OLLAMA_NUM_CTX, LLM_TIMEOUT_SECONDS	Model context window (8192) and per-call timeout (600 s).
AZURE_OPENAI_*	Endpoint, key, API version, chat deployment (gpt-4.1), embedding deployment.
AUTH_PROVIDER	local or keycloak. Can also be overridden from the shell (compose uses \${AUTH_PROVIDER:-local}).
SECRET_KEY	Random string used to encrypt the Keycloak tokens stored in sessions (openssl rand -hex 32). The old name JWT_SECRET is still accepted.
SESSION_IDLE_MINUTES, SESSION_MAX_HOURS	Session ends after 60 idle minutes or 12 hours in total.
SESSION_COOKIE_SECURE	true when the app is served over https (cookie then only travels over https).
APP_URL	The app's address as users open it; Keycloak sends users back to APP_URL/api/auth/callback.
KEYCLOAK_URL	Browser-facing Keycloak URL. The token issuer must equal KEYCLOAK_URL/realms/REALM.
KEYCLOAK_INTERNAL_URL	URL the backend container uses to talk to Keycloak (code exchange, keys, refresh, logout).
KEYCLOAK_CLIENT_SECRET	Only for a confidential client; empty for a public client (PKCE is always used).
KEYCLOAK_REALM, KEYCLOAK_CLIENT_ID	Realm and public client id (graphbase-app).
KEYCLOAK_ADMIN_USER/PASSWORD	Admin login of the optional local Keycloak container only.
COMPOSE_PROFILES	Optional services to start: ollama-proxy, ollama, keycloak.
BACKEND_PORT, FRONTEND_PORT	Host ports 8000 / 5173.

5. The Postgres database

All tables are created by `backend/app/migrations/001_init.sql`. To change the schema, add `002_something.sql`; `db.run_migrations()` applies files in name order, once each, recording them in `schema_migrations`.

Audit columns and the trigger

Every table has `created_at`, `updated_at` and `modified_by`. The app always sets `modified_by` (a user id, 'system' for automated jobs, 'seed'/cli' for commands). The trigger function `set_updated_at()` runs before every UPDATE: if any real column changed it sets `updated_at = now()`; if nothing really changed it keeps the old `updated_at` **and** the old `modified_by`, so a no-op save doesn't rewrite the audit. The users trigger is created with the argument 'last_login_at', so a login does not count as a modification of the user.

Tables

Table	Columns (besides the audit columns)	Notes
users	user_id PK, display_name, email, password_hash (null for Keycloak users), auth_source local keycloak, is_active, last_login_at	Created by CLI (local) or automatically on first Keycloak sign-in.
kb_catalog	kb_name PK, kb_type graph rag, domain, sub_domain, owner_id, status, status_detail, storage_ref, draft_schema jsonb, approved_schema jsonb, approved_cypher, approved_by, approved_at	One row per KB. kb_name must match <code>^[a-z][a-z0-9]{2,62}\$</code> . status: draft, extracting, awaiting_review, building, ingesting, ready, failed.
knowledge_bases	user_id, kb_name, kb_type, domain, sub_domain, access owner user; PK (user_id, kb_name)	As specified: one row per user per KB they can reach. Deleted on revoke.
kb_access	id, kb_name, user_id, role owner user, granted_by, granted_at, revoked_at	The audit trail. Owner row has <code>granted_by='system'</code> . Unique index allows only one active (revoked_at null) row per user per KB. Revoke sets <code>revoked_at + modified_by=revoker</code> ; rows are never deleted.
kb_pii_fields	id, kb_name, node_label, property_name, source_document, pii_category, sensitivity high medium low, confidence, occurrences, reason, detected_by, status	Graph rows: node_label + property_name. Document rows: source_document + occurrences. Never the raw value. status (detected confirmed dismissed) is reserved for a future review UI.
jobs	id, kb_name, job_type graph_extract graph_build rag_ingest add_data, status queued running succeeded failed cancelled, source_file, steps jsonb, progress, error, started_by, started_at, finished_at	Drives the Processing screen and upload chips.
pipeline_runs	id, kb_name, run_no, job_id, run_type initial_build add_data rag_ingest, source_file, status, rows_total loaded rejected, nodes_created, relationships_created, chunks_added, summary, rejected_report jsonb, started_by, started_at, finished_at	Shown on the Add data screen. run_no counts per KB (advisory lock prevents duplicates).
sessions (002)	id = SHA-256 of the cookie token, user_id, auth_source, expires_at, idle_expires_at, last_seen_at, revoked_at, revoked_reason, ip_address, user_agent, kc_refresh_token / kc_id_token (encrypted), kc_access_expires_at	Server-side sessions. The token itself is never stored. last_seen_at and idle_expires_at changes don't count as audited modifications.
oidc_login_requests (002)	state, code_verifier, nonce, next_path, expires_at	Pending Keycloak sign-ins (10 minutes). Deleted when used, so a sign-in response can't be replayed.

How the two spec tables work together: creating a KB inserts rows into `kb_catalog`, `knowledge_bases` (owner) and `kb_access` (owner, `granted_by='system'`) in one transaction. Granting adds a `kb_access` row and a `knowledge_bases` row. Revoking stamps `revoked_at` on the `kb_access` row and deletes the `knowledge_bases` row. `kb.role_of()` requires both an active `kb_access` row and a `knowledge_bases` row (and an active user), so the two tables can't disagree silently.

6. Authentication and sessions

Everything about sign-in and sessions happens in the backend. The browser never receives or stores a token: it only carries an `HttpOnly` cookie (JavaScript can't read it) containing a random session token. All code is in `backend/app/auth.py` and `backend/app/api/auth.py`.

Server-side sessions

- `create_session()` makes a 256-bit random token, stores its SHA-256 hash in the `sessions` table (never the token) with the user, sign-in type, IP, browser, an idle expiry and an absolute expiry, and sets the cookie `graphbase_session` with `HttpOnly`, `SameSite=Lax`, `Path=/` and `Secure` when `SESSION_COOKIE_SECURE=true`.
- `current_user()` is the FastAPI dependency on every protected route. It hashes the cookie, loads the session and the user, and refuses (401) if the session is unknown, revoked, past its absolute expiry or idle for longer than `SESSION_IDLE_MINUTES`, if the user was deactivated, or if the sign-in mode was switched. Using the app pushes the idle expiry forward (written at most once a minute).
- `POST /api/auth/logout` revokes the session in the database (`revoked_at`, `reason`, `modified_by`) and clears the cookie; an old copy of the cookie stops working immediately. Deactivating a user with the CLI ends all their sessions.
- **CSRF protection:** because sessions are cookies, every `POST/PUT/DELETE` to `/api` must carry the header `X-Requested-With: graphbase` (added by the frontend's `api.js`). Other websites can't send custom headers without a CORS preflight, which the API never allows. `SameSite=Lax` adds a second layer.

Local sign-in (testing)

- `POST /api/auth/login` checks the bcrypt hash and creates a session. A wrong password and an unknown user give the same message; unknown users are still checked against a dummy hash so timing doesn't reveal which user IDs exist. Passwords are limited to 72 bytes (bcrypt's limit).

Keycloak sign-in (production)

The backend runs the OpenID Connect authorization-code flow with PKCE itself:

- The Sign in button goes to `GET /api/auth/login?next=...`. The backend creates a random state, nonce and PKCE verifier, stores them in `oidc_login_requests` (10 minutes) and redirects the browser to Keycloak's login page at `KEYCLOAK_URL`.
- Keycloak sends the browser back to `APP_URL/api/auth/callback?code&state`. The backend deletes the pending request (so it can't be replayed), exchanges the code with Keycloak over `KEYCLOAK_INTERNAL_URL` using the verifier (and the client secret if one is set), and verifies the ID token: RS256 signature with Keycloak's published keys, issuer = `KEYCLOAK_URL/realms/REALM`, audience = client id, token type ID, and the nonce.
- `preferred_username` becomes the `user_id`; the users row is created on first sign-in and name/email refreshed later. The session stores Keycloak's refresh and ID tokens encrypted with `SECRET_KEY` (Fernet/AES), then the browser is redirected to the page it asked for (only same-site paths are allowed, so the login can't be used as an open redirect).
- When Keycloak's access token expires, the next request makes the backend refresh it with Keycloak. If Keycloak refuses (user disabled, signed out elsewhere, session ended), the app session is revoked too. If Keycloak is unreachable the API answers 503.
- Sign-out also ends the Keycloak session from the server (back-channel logout with the refresh token).
- An owner can only grant access to a user id that exists in the users table. For someone who has never signed in, an admin runs `python -m app.cli create-user NAME --name '...' --keycloak`.

`GET /api/auth/config` tells the frontend which sign-in screen to show. There is no JWT handling in the browser and no token in `localStorage` or `sessionStorage` (a browser test checks this).

7. Access control

backend/app/kb.py: `require_access(user, kb_name, roles)` is called at the top of every endpoint that touches a KB. It returns the catalog row plus the caller's role, or raises: 404 if the KB doesn't exist, 403 if the caller has no access, 403 with 'Only the owner ...' if their role isn't in `roles`. The UI hides buttons too, but the server never relies on that.

- `create()`: inserts the owner rows (chapter 5); 409 if the name exists, 422 if the name breaks the naming rule.
- `grant()`: owner only; the target must exist and be active (404 otherwise); 409 if they already have access; only the 'user' role can be granted (a KB has exactly one owner).
- `revoke()`: owner only; the owner can't be revoked (400); takes effect on the next request.
- `people()` and `access_log()` feed the Access screen. The log is rebuilt from `kb_access` rows: each owner row gives a 'created' event, each user row a 'granted' event, and each `revoked_at` a 'revoked' event (`actor = modified_by`).
- `list_for_user()` returns only KBs where the caller has an active grant.

8. Background jobs and pipelines

Extraction, building, ingest and add-data take seconds to minutes, so the endpoint creates a job and returns at once. `backend/app/jobs.py` runs jobs on a thread pool of 2 workers inside the backend process. State lives in the jobs table, so any request (and a restarted server) can read it.

- `create()` stores the job with its list of named steps (all 'waiting').
- `submit(job_id, fn, ...)` marks it running, calls `fn`, then marks it succeeded, or failed with the error message (and calls `on_error`, which usually sets the KB status to failed).
- `step(job_id, i, status, detail, progress)` updates one step; progress defaults to the share of completed steps. It raises `JobCancelled` if someone cancelled the job, which stops the work at the next step.
- `recover_interrupted()` (at start-up) fails running jobs/runs and resets stuck KB statuses.
- Limitation: jobs live in the API process. For many concurrent users you would move them to a queue (Celery/RQ).

What each job does (backend/app/pipelines.py)

Function	Steps
<code>graph_extract</code>	Read file -> <code>extraction.extract()</code> (LLM) -> store <code>draft_schema</code> , status <code>awaiting_review</code> -> <code>sync_graph_pii()</code> writes <code>kb_pii_fields</code> .
<code>graph_build</code>	Runs after Submit: read the file again -> <code>loader.plan_load()</code> -> <code>loader.execute_plan()</code> -> counts -> pipeline run 1 'initial_build' with the rejected-rows report -> status ready.
<code>graph_add_data</code>	Match the new file to the approved schema -> <code>plan_load</code> (merge or not) -> if bad rows and 'skip rows' is off, fail and write nothing -> execute -> pipeline run with report.
<code>rag_ingest</code>	Used for create and add-data: parse each file (one pipeline run per file) -> chunk -> embed + store -> PII scan per document -> status ready (or failed if nothing was readable).

Uploads are saved to the uploads volume as `/uploads/KB/xxxxxxxxx_filename` (max 50 MB, empty files refused). The original file is kept so extraction can be re-run and the build reads the full file.

9. Reading spreadsheets (tabular.py)

Real spreadsheets are messy, and the test dataset is messy on purpose. `read_table_file()` returns a list of `Sheet` objects (name, header_row, columns, rows with their file row number in `'_row'`, and a per-column profile).

- **XLSX** via `openpyxl` (every non-empty sheet). **CSV**: decoded as UTF-8 with or without BOM, falling back to cp1252; the delimiter is sniffed among comma, semicolon, tab and pipe; Windows line endings are fine.
- **Header detection** (`_find_header`): within the first 15 rows, the first row with at least `max(2, 60%` of the widest row) non-empty text cells that aren't all numbers. So a title row and a blank row above the header (as in the Suppliers sheet) are skipped.
- Blank rows are dropped; duplicate header names get suffixes; unnamed empty trailing columns are removed.
- Corrupt workbooks, empty files and header-only files raise `TabularError` with a message that is shown to the user.

Value parsing

Function	Accepts
<code>parse_number</code>	7, '42', '1,250', '12,000', 'INR 1250', 'Rs. 1250', currency symbols, '(1,200.00)' as -1200. Phone numbers stay text (they contain '+' or spaces).
<code>parse_date</code>	Excel dates, '2026-09-22', '22/09/2026' (day first), '22-09-2026', 'Sep 22, 2026', '22 Sep 2026', ISO with time.
<code>parse_bool</code>	Y/N, yes/no, TRUE/FALSE, T/F, 1/0.
<code>norm_key</code>	Key comparison form: strip + upper; 5100.0 becomes '5100'.
<code>norm_header</code>	Header comparison form: lower case letters and digits only ('Order ID' == 'order_id').

`infer_type()` picks boolean (all values are boolean words, not only 0/1), date (90% parse), integer/float (90% parse), otherwise string; values with leading zeros stay strings. The 90% threshold lets a few 'N/A' cells become null instead of turning a numeric column into text. `profile_column()` also records fill rate, distinct count/ratio, whether values look like IDs, and up to 5 samples (used in LLM prompts).

10. The graph schema and Cypher generation

The schema is the contract between the LLM, the Review screen and the loader. It is JSON, stored in `kb_catalog.draft_schema` (during review) and `approved_schema` (after Submit).

```
{ "source_file": "supplier_orders.xlsx", "source_path": "/uploads/...",
  "sheets": { "Suppliers": { "header_row": 3, "rows": 50, "columns": { "Supplier ID": {type, fill_rate, ...} } } },
  "nodes": [ { "id": "n1", "label": "Supplier", "sheet": "Suppliers", "role": "row", # or "embedded"
    "key": { "name": "supplier_id", "column": "Supplier ID" },
    "properties": [ { "name": "name", "column": "Supplier Name", "type": "string" } ], "count": 48 } ],
  "relationships": [ { "id": "r1", "type": "SUPPLIES", "sheet": "Products",
    "from": { "label": "Supplier", "column": "Supplier" }, "to": { "label": "Product", "column": "SKU" },
    "properties": [], "required": true, "count": 307 } ],
  "pii": [ { "sheet": "Customers", "column": "Email", "category": "email", "sensitivity": "medium", ... } ] }
```

- **role**: 'row' = each row of the sheet is one of these (a missing key rejects the row); 'embedded' = an entity named in a column of another entity's sheet (e.g. Vendor in a ledger; a blank just skips it).
- **relationship columns**: 'from.column' and 'to.column' are the columns in the relationship's sheet holding each end's key value. **required**: if false, an empty cell just means 'no relationship' for that row (extraction sets it from the fill rate: required when 90%+ filled).
- **normalize()** repairs names the user typed: labels to PascalCase, properties to snake_case, relationship types to UPPER_SNAKE, and renames follow into relationships. **validate()** lists problems (duplicate labels, reserved KB_labels, missing columns, relationships to unknown labels). **clean()** = both, raising SchemaError.
- **Cypher is generated from the schema, never copied from the LLM.** `node_cypher()` and `relationship_cypher()` produce UNWIND \$rows batches; `preview()` produces the readable one-line statements shown on the Review screen. Same code, so the preview is what runs.

```
UNWIND $rows AS row
MERGE (n:`Supplier`:`KB_retail_supply_chain_kg` {supplier_id: row.supplier_id}) # KB label only in single mod
SET n.name = row.name, n.country = row.country, ...
```

```
UNWIND $rows AS row
MATCH (a:`Supplier`:`KB_...` {supplier_id: row.from_key})
MATCH (b:`Product`:`KB_...` {sku: row.to_key})
MERGE (a)-[r:`SUPPLIES`]->(b)
SET r.some_prop = row.some_prop
```

`index_cypher()` creates an index on each label's key property before loading. `describe_for_llm()` writes the schema as text for the chat prompt.

11. The LLM factory (llm.py)

Only this file knows which LLM provider is used. Everything else calls:

- `get_llm(json_mode=False, temperature=0)`: ChatOllama with the configured model, `num_ctx`, a 30-minute `keep_alive` and the timeout; `json_mode` sets Ollama's JSON output format.
- `get_embeddings()`: OllamaEmbeddings. For `nomic-embed-text` it adds the task prefixes 'search_document: ' / 'search_query: ' (`_PrefixedEmbeddings`); that model needs them for good retrieval.
- `ask_json(system, user)`: calls the model and parses one JSON object (`parse_json` tolerates code fences and prose around the JSON). If the reply isn't valid JSON it retries once, telling the model what was wrong.
- `ask_text(system, user)`: plain text answer.

Switching to Azure OpenAI GPT-4.1

- Uncomment the `from langchain_openai import ...` line and the two `# if s.llm_provider == "azure":` blocks in `llm.py`.
- Uncomment `langchain-openai==1.6.6` in `backend/requirements.txt` and rebuild the backend.
- Set `LLM_PROVIDER=azure` and the `AZURE_OPENAI_*` variables. Nothing else changes.
- Note: switching embedding models means existing Chroma collections were embedded with the old model; re-ingest RAG documents after switching.

12. Knowledge-graph extraction (extraction.py)

This is the most important and most defensive part of the code. The rule is '**the LLM proposes, the evidence verifies**': the model makes the modelling decisions, deterministic profiling of the data supplies evidence in the prompt and checks every answer, and anything unusable falls back to a heuristic so the user always gets an editable draft. This was necessary because qwen2.5:3b, in real runs, listed every column as 'ignore', reused the label 'Customer' for orders, modelled the Inventory sheet as an entity keyed by stock_qty, reversed directions and flagged company names and dates as PII. The checks are cheap and should stay with bigger models.

The LLM is sent each sheet's column names, types, fill rates, distinct counts and a few sample values - never the whole file. Large files therefore cost the same as small ones.

Step 2 - what each sheet is (one LLM call per sheet)

- `reference_hints()` computes evidence first: which columns contain another sheet's IDs (at least 60% of values overlap that sheet's unique key) and which columns are unique on every row (distinct ratio 95%+). These go into the prompt.
- `ask_sheet_entities()` asks for: the row entity (label, key column, properties) or null for a link table, reference columns, embedded entities, and ignore columns.
- `build_nodes()` cleans the answers: column names are matched case-insensitively; a key must be text (or an integer code, never a quantity or date: `_key_ok`); 'ignore' is only honoured for junk columns (`_looks_junk`: unnamed, essentially empty, or a running row counter like `row_no`); 'link table' is only accepted if the evidence shows 2+ reference columns.
- If two sheets claim the same label, the sheet whose key is most unique keeps it; the other gets an entity named after itself. A sheet whose key isn't unique but matches another sheet's IDs is demoted to a link table (Inventory keyed by sku). A sheet left without an entity but with its own ID column (`order_id`) gets an entity named after that column (`_entity_from_id_column`).

Step 3 - relationships

- `detect_links()` finds relationships from data, not from the LLM: a column whose values overlap 60%+ with another node type's keys is a reference. Self-references work (Substitute SKU -> Product).
- Line-level sheets: if the row key repeats (several rows per order), `_line_level_columns()` finds columns that vary within one order (sku, qty, notes). Those are moved from the Order node to the Order->Product relationship, so qty becomes a relationship property. Values are compared after parsing, so one date written four ways is not 'varying'.
- A link table with two references becomes one relationship with the remaining columns as its properties (Product-[STORED_IN {stock_qty, last_counted}]->Warehouse).
- `name_links()` asks the LLM to write a sentence ('Supplier supplies Product'), a type and the direction for each link. Its answer must use the link's two node types, else the default direction is kept; long names are cut back to the verb (`_short_rel`); failures fall back to `HAS_X`.

Step 4 - Cypher, and step 5 - PII

- The schema is normalised and validated; invalid relationships are dropped rather than failing the draft.
- `detect_pii()` asks the LLM to classify **every** stored column (person_name, email, phone, address, date_of_birth, government_id, bank_account, financial, free_text, or none). Asking it to classify every column works far better with small models than asking it to list PII.
- `verify_pii()` checks each claim against the values: a date of birth must be 16+ years ago (or the column named DOB); a government ID/bank account must match a pattern or column-name rule; an address needs digits and 3+ words; a person name must be 2-5 words without digits and without company words (Ltd, Textiles, GmbH...). Claims under 50% confidence are dropped.
- `rule_pii()` adds what the model missed: e-mail, phone, PAN/Aadhaar patterns, column names (bank/IBAN/IFSC, DOB, PAN/passport, address) and free-text columns containing e-mails or phone numbers. Such rows have `detected_by='rules'`; rows the LLM found have `detected_by='llm'`.

- `pii_targets()` maps PII columns to the current node/relationship property names, so renames on the Review screen follow through to `kb_pii_fields`.

13. Review and Submit

- GET `/api/kbs/KB/review` (owner): the draft schema, the Cypher preview, a summary (node types, entities, relationship types) and the PII list.
- POST `.../review/preview`: the frontend sends the edited schema (debounced as you type) and gets back the new Cypher, or a 422 listing problems. Nothing is saved.
- PUT `.../review`: saves the edited draft and re-syncs `kb_pii_fields`.
- POST `.../submit`: cleans the schema, stores it as `approved_schema` with `approved_cypher`, `approved_by` and `approved_at`, sets status 'building' and starts `graph_build`. This is the first time anything is written to Neo4j.
- `_merge_edit()`: the client can only change nodes, relationships and pii; sheet profiles and the source file path always come from the server copy, so a user can't point the loader at another file.

14. Loading rows into Neo4j (loader.py)

The same code builds the graph after Submit and loads additional data later. `plan_load()` validates everything in memory first; `execute_plan()` writes.

Rules (the 'ingest contract', also written in `data/generate_dataset.py`)

- **Sheets are matched by columns** (`match_sheets`): a file sheet matches a schema sheet if it contains all its required columns (row-entity keys and relationship end columns), compared with `norm_header`. So the October file's 'Order ID', 'Customer ID' headers in a different order still match 'order_id'.
- **Keys** are compared with `norm_key` (strip + upper), so ' sup-011' and 'SUP-011 ' are the same supplier.
- **A row is rejected** (all or nothing) if: a row-entity key is empty ('missing order_id'); a required relationship column is empty ('missing customer_id'); a referenced entity doesn't exist in the graph or in accepted rows ('unknown Product: SKU-77777'); or, with merge turned off, its key already exists ('already exists').
- **Rejections cascade**: a rejected product's SKU isn't available to other rows. `plan_load` repeats validation until nothing changes (at most 10 passes).
- **Merge on key**: MERGE on the key property; repeated rows update properties, the last row wins. Embedded entities don't let a blank cell wipe a known value.
- **Stored key spelling**: the spelling already in the graph, else the most common spelling in the file; ID-like keys (no spaces, contain a digit) are upper-cased.
- **Values are typed** by `coerce()`: numbers, dates (stored as Neo4j DATE), booleans; unparseable cells become null.
- **Order**: all nodes first, then relationships, so a row may reference a product defined later in the file.
- Only properties whose columns exist in this file are SET, so a file missing an optional column doesn't wipe it.
- Writes are batched 1,000 rows per query; the report keeps up to 5,000 rejected rows (sheet, file row, reason).

Result on the test workbook: exactly the expected 48 suppliers, 307 products, 9 warehouses, 120 customers, 1,607 orders, all relationship counts, and 32 rejected rows with the right reasons (see chapter 22).

15. Neo4j modes and query safety (graphstore.py)

All Neo4j access for a KB goes through `GraphStore(kb_name)`.

	NEO4J_MODE=single (Community)	NEO4J_MODE=multi (Enterprise)
Where a KB lives	The default database; every node has an extra label <code>KB_kbname</code>	Its own database: kb name with '_' replaced by '-' (database names can't contain underscores)
Create	Nothing to create	CREATE DATABASE `name` IF NOT EXISTS WAIT (on system)
Writes	Generated Cypher includes the KB label	Session bound to the KB database
Chat queries	Rewritten by <code>scope_cypher()</code> so every node pattern has the KB label	Session bound to the KB database; USE is forbidden
storage_ref	label:KB_name	database:name

Chat Cypher is treated as untrusted (four layers)

- `check_read_only()` (after hiding string literals and comments): one statement only; refuses CREATE, MERGE, DELETE, DETACH, SET, REMOVE, DROP, LOAD CSV, CALL, FOREACH, USE, ALTER, GRANT, REVOKE, DENY, START, STOP, TERMINATE, SHOW, RENAME, ENABLE, DISABLE, FINISH, INSERT and any `KB_label`.
- `scope_cypher()` (single mode): adds the KB label to every node pattern in a pattern position - MATCH, OPTIONAL MATCH, paths, EXISTS/COUNT subqueries, pattern comprehensions, shortestPath. Label expressions like `A|B` become `(A|B)&KB_x`. If a pattern can't be scoped safely (e.g. WHERE inside a node pattern with nested brackets), the query is refused rather than risking a leak.
- The query runs in a **read transaction** in a READ session, so Neo4j itself rejects writes even if the text check were bypassed (a test proves this).
- A 30-second timeout and a 200-row limit.

16. Adding data (pipeline 7)

- POST `/api/kbs/KB/add-data/check` (owner or user): reads the file and reports which sheets match which schema sheets and which columns are unused. The UI shows 'Schema matched' or 'No match'.
- POST `/api/kbs/KB/add-data` with `merge_existing` and `skip_invalid` (the two checkboxes). Graph KBs: one file; starts `graph_add_data`. RAG KBs: one or more documents; starts `rag_ingest`.
- Merge unticked: rows whose key already exists are rejected instead of merged.
- Skip unticked: if any row is bad, the run fails and nothing is written (the report still lists the bad rows).
- Every load is a pipeline run with numbers and a report: GET `.../runs` and GET `.../runs/ID/report`.
- Verified numbers: the October file adds exactly 476 orders and 1,200 order lines, merges the 20 lines already loaded and rejects 10 rows (4 unknown customers, 3 unknown SKUs, 3 missing order ids); the warehouse CSV (BOM, semicolons) adds 14 stock pairs, updates 30 and rejects 12.

17. RAG documents (rag.py)

- **Parsing** (`extract_text`): PDF with `pypdf` page by page, removing lines that repeat on 60%+ of pages (headers and footers; digits are ignored when comparing so 'Page 3' matches 'Page 4'); DOCX with `python-docx`, paragraphs and table rows in document order (cells joined with ' | '); TXT decoded as UTF-8 or cp1252 with line endings normalised. Scanned PDFs without text, empty files and unsupported types raise `DocumentError`.
- **Chunking**: `RecursiveCharacterTextSplitter`, 900 characters with 150 overlap, keeping the page number.
- **Storing** (`store_chunks`): one Chroma collection per KB (named after the KB, cosine distance). Earlier chunks of the same file are deleted first, so re-uploading replaces instead of duplicating. The file name is embedded together with the chunk text so questions naming a document find it. Metadata: source, page, chunk, run_id.
- **Retrieval** (`retrieve`): embeds the question and returns the top k passages with source, page and a similarity score.
- **PII scan** (`scan_pii`): e-mails, phone numbers, PAN/Aadhaar numbers and 'date of birth' mentions by pattern; person names by showing the LLM the capitalised phrases found in the document and asking which are people (small models classify much better than they extract); the model's picks must be among the phrases. Only counts per category are stored, never the values.

18. Chat (chat.py and cypher_repair.py)

Graph KBs: a LangGraph flow

```
generate --> run --(error and fewer than 2 attempts)--> generate
\-----> answer --> END
```

- **generate**: the prompt holds the schema text (labels, properties with types, example ids and up to 12 real values of low-cardinality text properties such as cities and statuses), worked examples generated from this KB's own schema (`examples()`), the last 3 turns of the conversation (question, Cypher, answer) for follow-ups, and on a retry the previous query and its error. The reply is JSON `{cypher}`. The schema text is cached per KB until the approved schema changes.
- The Cypher then goes through `repair()`: label/type spelling mapped to the schema (`PaidTo` -> `PAID_TO`), arrows turned to the schema's direction, properties moved between node and relationship when they obviously belong to the other (`w.stock_qty` -> `r.stock_qty`), and inline maps moved to the right node.
- **run**: `lint()` checks the query against the schema (unknown labels/types, a relationship between the wrong labels, unknown properties). Problems go back to the model as precise feedback for one retry. Then `run_readonly()` (chapter 15). Neo4j errors get plain-language hints (`error_hint`, e.g. 'use ORDER BY ... DESC LIMIT 1, not max() in WHERE').
- **answer**: refused -> 'I can only read from this knowledge base'; still failing -> 'couldn't turn that into a working query'; no rows -> 'No matching data was found' (no LLM call, so no invented answer); otherwise the LLM answers from at most 60 result rows.
- Response: answer, cypher, path (chips from the first MATCH pattern, e.g. Supplier SUPPLIES Product STORED_IN Warehouse: Chennai), first 20 rows, row count, error.

RAG KBs

`rag_answer()` retrieves 6 passages (for short follow-up questions the previous question is added to the search text), numbers them with their source and page, and asks the LLM to answer only from them, quote exact figures, prefer current rules over superseded ones and cite [n]. The sources are returned for the UI.

Access

POST `/api/kbs/KB/chat` calls `require_access()` on every question and requires status ready. Chat history is kept in the browser and sent with each question (last 5 turns, trimmed); it isn't stored on the server.

19. API reference

Method and path	Who	What
GET /api/health	anyone	Status of Postgres, Neo4j, Chroma, Ollama
GET /api/auth/config	anyone	local or keycloak (+ URL, realm, client id)
POST /api/auth/login	anyone (local mode)	user_id + password -> session cookie
GET /api/auth/login	anyone (Keycloak mode)	Redirect to Keycloak's login page
GET /api/auth/callback	Keycloak redirect	Finish Keycloak sign-in, set session cookie
POST /api/auth/logout	anyone	Revoke the session (and the Keycloak session)
GET /api/auth/me	signed in	Current user
GET /api/kbs	signed in	KBs the caller can reach, with role, status and last job
POST /api/kbs	signed in	Create: kb_name, kb_type, domain, sub_domain, files -> job id (caller becomes owner)
GET /api/kbs/KB	owner, user	Details, counts/stats, last job
POST /api/kbs/KB/extract	owner	Re-run extraction (failed or awaiting review graph KBs)
GET /api/jobs/ID	owner, user of the job's KB	Job progress and steps
POST /api/jobs/ID/cancel	owner or the person who started it	Cancel a running job
GET /api/kbs/KB/review	owner	Draft schema, Cypher preview, summary, PII
POST /api/kbs/KB/review/preview	owner	Cypher for an edited schema (not saved)
PUT /api/kbs/KB/review	owner	Save the edited draft
POST /api/kbs/KB/submit	owner	Approve and build the graph
GET /api/kbs/KB/pii	owner, user	kb_pii_fields rows (no raw values)
GET /api/kbs/KB/access	owner	People with access + access log
POST /api/kbs/KB/access	owner	Grant {user_id, role:'user'}
DELETE /api/kbs/KB/access/USER	owner	Revoke
GET /api/users/USER	signed in	Check a user id exists (name only)
POST /api/kbs/KB/add-data/check	owner, user	Does this file match the schema?
POST /api/kbs/KB/add-data	owner, user	Load a file / documents (KB must be ready)
GET /api/kbs/KB/runs	owner, user	Pipeline runs
GET /api/kbs/KB/runs/ID/report	owner, user	Rejected rows of a run
POST /api/kbs/KB/chat	owner, user	{question, history} -> answer (+ cypher/path/rows or sources)

Every POST/PUT/DELETE also needs the header X-Requested-With: graphbase (CSRF). Status codes: 401 not signed in / expired session; 403 no access or wrong role; 404 unknown KB/job/user; 409 conflicts (name taken, already granted, wrong status); 413 file too large; 415 wrong file type; 422 invalid input or schema. Interactive docs: <http://localhost:8000/docs>.

20. The React frontend

Route	File	Screen
/login	pages/Login.jsx	Mockup 1. Local mode: user ID + password form. Keycloak mode: Sign in goes to /api/auth/login (backend redirects to Keycloak).
/workspace	pages/Workspace.jsx	Mockup 2. Upload cards, Create form (switches graph/RAG), KB list with role badges and the right action button; polls while jobs run.
/kbs/KB/jobs/ID	pages/Processing.jsx	Mockup 3. Progress bar and steps; auto-opens Review (or Chat after a build).
/kbs/KB/review	pages/Review.jsx	Mockup 4. Edit/Delete/Undo nodes and relationships, add new ones, PII badges, live Cypher, pending-changes footer, Discard, Submit.
/access	pages/Access.jsx	Mockup 5. KB selector (owned KBs), grant form, people table, access log.
/add-data	pages/AddData.jsx	Mockup 6. KB selector, stats, drop zone, schema check, options, pipeline runs, rejected-rows report.
/chat	pages/Chat.jsx	Mockup 7. KB list with search, conversation, Cypher block, path chips, sources.

- `App.jsx`: routes; Protected asks the backend who is signed in (`/api/auth/me`) and sends the user to `/login` if the session isn't valid.
- `api.js`: fetch wrapper that sends the session cookie and the CSRF header, turns error responses into readable messages and sends the user to `/login` on 401.
- `auth.js`: calls the backend's sign-in endpoints. Local mode posts the password; Keycloak mode simply navigates to `/api/auth/login`. Sign-out posts to `/api/auth/logout`. Nothing is stored in the browser.
- `components/common.jsx`: top bar, badges, drop zone, polling hook, error box, modal. `styles.css`: colours, fonts and components copied from the mockup HTML.
- `vite.config.js`: proxies `/api` (including the Keycloak callback) to the backend and allows access by machine name. `eslint.config.js`: lint rules (`npm run lint`).

21. Admin commands and demo data

```
docker compose exec backend python -m app.cli migrate
docker compose exec backend python -m app.cli create-user meera.s --name "Meera S" [--email ..] [--password ..]
docker compose exec backend python -m app.cli create-user meera.s --name "Meera S" --keycloak # no password
docker compose exec backend python -m app.cli set-password meera.s
docker compose exec backend python -m app.cli deactivate-user meera.s
docker compose exec backend python -m app.cli seed-demo-users # 6 users, password test1234
docker compose exec backend python -m app.cli seed-demo-data [--llm-pii]
```

seed-demo-data (app/demo.py) builds the mockup KBs from data/samples through the same pipeline functions, but with reviewed schemas instead of waiting for the LLM: retail_supply_chain_kg (owner priya.nair; arjun, sneha, karthik have access), finance_ledger_kg (owner arjun.mehta; priya has access), retail_policies_rag (owner priya) and supplier_orders_review_kg (a draft waiting on the Review screen). It is idempotent. PII uses rules only unless --llm-pii is given. Don't run it in production.

22. The test dataset

`data/generate_dataset.py` generates the files with a fixed random seed (same output every time) and writes the correct answers to `data/expected/manifest.json`. It is deliberately hard, so the tests prove the code handles real-world mess.

File	Traps
supplier_orders.xlsx (6 sheets, ~6,300 rows)	Suppliers header on row 3 under a title; IDs with spaces/lower case; duplicate rows; prices like 'INR 1250', '1,250.00' with currency symbols; four date formats and real Excel dates; Y/yes/TRUE/1 booleans; 'N/A' ratings; different column names for the same thing across sheets; 5 products whose supplier is written as a name (rejected); 27 bad order rows (missing customer, unknown SKU including a letter O instead of zero, unknown warehouse, missing order id); forward references; repeated stock rows (last wins); blank rows; a junk row_no column; PII hidden in free-text notes; company names that look like person names.
supplier_orders_october.xlsx	Renamed and reordered headers; 20 already-loaded lines; 10 bad rows.
warehouse_update.csv	UTF-8 BOM, semicolons, CRLF; 12 rows with no SKU.
finance_ledger.csv	Second KB for isolation tests; accounting negatives '(1,200.00)'; vendors with the same names as retail suppliers.
returns_policy.pdf / vendor_handbook.docx / warehouse_sop.txt	A fact only inside a PDF table; an outdated 45-day rule next to the current 30-day rule; repeated headers/footers; a DOCX table; Tamil text; contact names, e-mails and phones (PII).
edge_cases/	A corrupt .xlsx and a header-only CSV.

The manifest also holds 22 questions with their correct answers: 11 retail graph (1 stretch: 'Bengaluru' vs the stored 'Bangalore'), 2 finance graph, 6 RAG, 3 security.

23. Tests and results

Suite	Command	Covers
Backend (fast)	<code>docker compose exec backend pytest -q</code>	173 tests, about 4 minutes. Uses a throwaway database <code>graphbase_test</code> and KB names starting with <code>t_</code> . The LLM is scripted where needed; Neo4j, Postgres, Chroma and the embedding model are real.
LLM benchmark	<code>docker compose exec backend pytest -m llm -s</code>	Real model on the dataset: extraction, graph chat, RAG chat, document PII. Writes <code>backend/tests/reports/llm_quality.json</code> .
Browser	<code>.venv/bin/python -m pytest e2e -q</code>	11 Playwright tests of every screen against the running stack (needs <code>seed-demo-data</code>). Screenshots in <code>e2e/screenshots/</code> .

What the fast tests check

File	Checks
<code>test_phase1_health.py</code>	All services reachable.
<code>test_phase2_schema_auth.py</code>	Audit columns, trigger behaviour, one active grant, name rules; sessions: <code>HttpOnly</code> cookie, only the hash stored, idle/absolute expiry, activity extends idle time, logout revokes, deactivation ends sessions, CSRF header, provider switch; Keycloak (real local Keycloak): PKCE redirect, bad callbacks, open-redirect guard, user provisioning, encrypted tokens, ID-token checks, re-check/refresh, back-channel logout.
<code>test_phase3_llm.py</code>	Factory, provider switch, JSON parsing and retry, nomic prefixes, real embeddings.
<code>test_units.py</code>	Number/date/boolean parsing, header detection, CSV formats, bad files, Cypher scoping (11 patterns), 12 refused write/escape queries, multi-database mode (mocked Enterprise), schema editing, Cypher repair of real model mistakes, lint and error hints.
<code>test_phase4_extraction.py</code>	Extraction with a scripted LLM that makes the real model's mistakes; result loads to the exact manifest counts; garbage LLM still gives a valid draft; PII false positives rejected.
<code>test_phase4_loader.py</code>	Real Neo4j build: exact node/relationship counts, rejection reasons, typed values, last row wins, add-data numbers, strict mode, KB isolation, read-only at two levels.
<code>test_phase5_rag.py</code>	Parsing, header/footer removal, re-upload replaces, retrieval finds each fact, PII counts.
<code>test_phase6_api.py</code>	Whole flow through HTTP, access matrix (outsider/user/owner on every endpoint), audit rows, validation, bad files, add-data strict mode, RAG flow, jobs, restart recovery.

Latest results

Test	Result
Backend fast suite	173 passed (all non-LLM backend tests, with a real Keycloak for the sign-in tests); ruff and ESLint clean
Browser tests without the LLM (incl. Keycloak redirect login)	11/11 passed, including Keycloak sign-in run by the backend, 3 flows with the real LLM, and a check that the browser holds no session data
LLM extraction (qwen2.5:3b)	5/5 entity keys, 6/6 relationships, PII recall 1.0, precision 1.0, 400 s
LLM graph chat	core 6/11, stretch 0/1; delete request refused: yes
LLM RAG chat	6/6; handbook PII <code>{'email': 2, 'phone': 2, 'person_name': 1}</code>
Unseen domain (school workbook, real LLM)	keys 3/3, Enrolments recognised as link table, PII <code>['Students.Email', 'Students.Student Name', 'Teachers.Full Name']</code> ; relationship names swapped by the model (Course-ENROLLED_IN->Teacher, Student-TEACHES->Course) - fix on Review
Graph chat, previous run	7/11 core (same code except one prompt line) - 3b results vary by about one question per run

Question	OK	Answer (start)
How many suppliers are there?	yes	48
Which suppliers deliver products stored in the Chennai warehouse?	yes	Bluepeak Textiles, Kaveri Packaging, Suvarna Foods, Orion Components

Question	OK	Answer (start)
Which of those have a rating below 4?	no	Kaveri Packaging
Which customer ordered the largest total quantity?	yes	Ravi Gupta
How many orders were cancelled?	yes	115
Which warehouse has the largest capacity?	no	I couldn't turn that question into a working query. Try rephrasing it, for example by naming the entity you're
What is the substitute product for SKU-10007?	no	Tawa Eco
What is the total stock of SKU-10001 across all warehouses?	yes	3936
How many products does Bluepeak Textiles supply?	no	No products supplied by Bluepeak Textiles.
How many products are stored in the Bengaluru warehouse?	no	No products stored in the Bengaluru warehouse.
What is the total amount posted to cost centre CC-IT?	yes	9926365.97
Which vendor has the highest total spend?	no	No matching data was found in this knowledge graph.

All prompts and rules are domain-neutral: dataset-specific examples and word lists were removed on 30 Sep and every result above was measured after that. With gwen2.5:3b on a CPU laptop, extraction and RAG are strong; graph chat answers 6-7 of 11 core questions (the benchmark's 60% bar is met on some runs, not others). Re-run ``docker compose exec backend pytest -m llm -s`` with the work system's model before relying on graph chat.

24. Deployment and operations

- Setup steps for your workspace are in README.md ('Switching to production') and in the chat where this guide was delivered: clone, copy .env.example to .env, set passwords and SECRET_KEY, point OLLAMA_BASE_URL at Ollama, pull the models, docker compose up -d --build, check /api/health, then switch AUTH_PROVIDER to keycloak.
- **Keycloak client:** standard flow on, PKCE S256, redirect URI APP_URL/api/auth/callback. Public client, or confidential with KEYCLOAK_CLIENT_SECRET set. The browser never talks to Keycloak's token endpoint, so no web origins are needed.
- **Neo4j Enterprise:** NEO4J_IMAGE=neo4j:5-enterprise, NEO4J_ACCEPT_LICENSE=yes, NEO4J_MODE=multi - before creating real KBs (single-mode KBs are not migrated).
- **Logs:** docker compose logs backend. Health: GET /api/health. Auth mode: GET /api/auth/config.
- **Backups:** the volumes pgdata, neo4jdata, chromadata and uploads hold all state.
- **Dev-mode note:** the frontend runs the Vite dev server and the backend runs uvicorn with --reload. That is fine for an internal pilot; for a hardened production deployment build the frontend (npm run build) behind nginx with https (then set SESSION_COOKIE_SECURE=true) and drop --reload.
- **Secrets:** .env is not in git. The repository contains only test passwords (postgres/postgres, neo4j123 in .env.example, test1234 in the local Keycloak realm).

25. Questions you may be asked

Does the whole spreadsheet go to the LLM?

No. Only column names, types, fill rates, distinct counts and up to 5 sample values per column (tabular.profile_column). All rows are loaded by the loader using Cypher generated from the approved schema. This keeps cost and time independent of file size.

Is anything written to Neo4j before the user submits?

No. Extraction only stores a draft in kb_catalog.draft_schema. graph_build, which writes to Neo4j, is started only by POST /submit.

Why isn't the LLM's Cypher used directly to build the graph?

So the preview on the Review screen is exactly what runs, so user edits can't produce broken Cypher, and so an LLM can never write arbitrary statements. graph_schema.py generates the Cypher from the reviewed schema.

How do you stop chat from changing or deleting data?

Four layers: a keyword/statement check, KB scoping, a Neo4j read transaction in a READ session (Neo4j itself refuses writes), and a timeout plus row limit. Tested: 'delete all suppliers' is refused and the graph is unchanged.

How can one KB's user not see another KB's data in Neo4j Community?

Every node carries a KB_name label and scope_cypher adds that label to every node pattern in chat queries; unscopable queries are refused. Tests put two KBs with overlapping names (Bluepeak Textiles in both) in one database and prove each only sees its own data. On Enterprise each KB has its own database.

Where is access enforced?

On the server, in kb.require_access(), at the top of every endpoint that touches a KB. The test test_access_control_matrix calls every endpoint as an outsider, a user and the owner.

What does a 'user' get compared with the owner?

Chat and add data. Only the owner reviews, submits, re-runs extraction, grants, revokes and sees the access page.

How is the audit trail kept?

kb_access rows are never deleted: grants add rows (granted_by, granted_at), revokes set revoked_at and modified_by. Every table has created_at/updated_at/modified_by, with updated_at maintained by a trigger that ignores no-op updates and login timestamps.

How is PII detected and what is stored?

Spreadsheets: the LLM classifies every stored column, each claim is checked against the values, and pattern/name rules fill gaps. Documents: patterns for e-mails, phones and IDs, the LLM picks which capitalised phrases are people. kb_pii_fields stores the category, sensitivity, confidence, reason and (for documents) a count - never the values. It is shown as badges on the Review screen and via /api/kbs/KB/pii.

Does PII detection mask data in chat?

No. It records and flags PII (as agreed). Masking in answers would be a new feature: kb_pii_fields already says which properties to mask.

What happens to bad rows?

They are rejected per row with a reason and listed in the pipeline run report (sheet, file row number, reason). With 'Skip rows' unticked, one bad row makes the whole load fail with nothing written.

How are duplicates handled?

Nodes MERGE on their key (compared ignoring case and spaces), so duplicates merge and the last row wins. With 'Merge' unticked, rows for existing keys are rejected instead.

Can the same file be loaded twice?

Yes; it merges, so no duplicates are created (tested with 20 repeated lines).

What if extraction gets the model wrong?

The owner fixes it on the Review screen (rename, delete, add node types and relationships) with a live Cypher preview, or clicks 'Re-run extraction'.

How do I switch to Azure OpenAI?

Uncomment the Azure code in `llm.py` and `langchain-openai` in `requirements.txt`, set `LLM_PROVIDER=azure` and the `AZURE_OPENAI_*` values, rebuild. Re-ingest RAG documents if the embedding model changes.

How do I switch to Keycloak?

Configure the client (chapter 24), set `AUTH_PROVIDER=keycloak` and the `KEYCLOAK_*` values, docker compose up -d. Users appear on first sign-in; pre-provision with `create-user --keycloak`.

Why two Keycloak URLs?

`KEYCLOAK_URL` must match the issuer inside the tokens (what the browser uses).

`KEYCLOAK_INTERNAL_URL` is how the backend container reaches Keycloak to fetch signing keys; inside Docker that address can differ (e.g. `host.docker.internal`).

What happens if the server restarts during a job?

`recover_interrupted()` marks the job and run failed and the KB 'failed' with 'Interrupted by a server restart'. The owner can re-run extraction; the upload is kept.

Why is the LLM slow locally?

The laptop has 7 GB RAM and no GPU, so `qwen2.5:3b` runs on the CPU (30-90 s per call). Extraction of the 6-sheet workbook takes 7-10 minutes. A GPU or a hosted model makes this seconds.

How accurate is chat?

See chapter 23. Graph chat accuracy depends heavily on the model; the repair and lint layers raised `qwen2.5:3b` from 3/11 to 7/11 on the core questions. RAG chat scored 6/6. Re-run the benchmark with the work system's model: `docker compose exec backend pytest -m llm -s`.

Can a KB be deleted?

Not from the UI or API yet (deliberately, so the audit trail isn't lost). `GraphStore.drop()` and `rag.drop_collection()` exist for tests and could back a soft-delete feature.

How many users can it handle?

Fine for a team pilot. Limits: 2 background job workers in the API process, the Vite dev server, and the LLM's speed. Scale by moving jobs to a queue and serving a built frontend.

Where do I add a new database column or table?

A new file `backend/app/migrations/002_*.sql`; it is applied on the next start. Remember the audit columns and the `set_updated_at` trigger.

How do I add another node or relationship type to an existing graph?

Today the approved schema is fixed once built; `add-data` uses it. Changing it means creating a new KB (or adding a 'schema change' feature later).

Is anything hardcoded for the test data?

No. The app code has no dataset-specific logic: prompts use neutral examples (employees, invoices, patients), and the rules use generic signals (value overlap between sheets, uniqueness, fill rate, ID/e-mail/phone patterns, legal-form words like `Ltd/Pvt/GmbH`, column headers like 'Company' or 'Vendor'). Only `app/demo.py` (the optional demo seeder) and the tests know the sample files. A school workbook the code had never seen was extracted correctly both with the real model and with the LLM switched off.

Will it work for any CSV/XLSX?

Any file where each sheet is a table with one header row: one row per record (or per line item), plus columns that reference other sheets by ID. Handled: header rows below titles, blank rows, messy IDs, currencies, date formats, booleans, BOM/semicolon CSVs, junction sheets, multiple entities in one sheet. Not handled automatically: old .xls files (save as .xlsx), several tables in one sheet, headers spread over two rows, pivoted/cross-tab layouts (months as columns), merged cells, composite keys (two columns together), references by name when the other sheet is keyed by code, month-first dates like 03/04/2026 (read as 3 April), formulas without saved values, files over 50 MB. Anything the extraction misses can be fixed on the Review screen.

Is authentication done in the backend? Where is the session?

Yes, entirely. Sessions are rows in the Postgres sessions table; the browser only holds an HttpOnly cookie with a random token, and nothing is kept in browser storage. The backend checks the session on every request, expires it when idle, revokes it on sign-out, and for Keycloak runs the whole sign-in flow and re-checks the user with Keycloak.

Do I need a JWT or anything from Keycloak's admin besides the client?

No. Keycloak's tokens are handled by the backend. You need the Keycloak URL, realm, a client id (and secret if the client is confidential) and the redirect URI registered. SECRET_KEY in .env is any random string you generate (openssl rand -hex 32).

Why the X-Requested-With header?

Cookie sessions need CSRF protection. The API rejects state-changing requests without that header; other websites can't add it, so they can't act as a signed-in user.

Does the code follow standards?

Python is formatted and linted with ruff (backend/ruff.toml: pycodestyle, pyflakes, bugbear, import order, pyupgrade, simplify) and React with ESLint (recommended, react, react-hooks); both report no issues. Known structural improvements are listed in chapter 24 (job queue, typed schema models, splitting the two largest modules).

What is 'role: embedded' in the schema?

An entity that has no sheet of its own and is named in a column of another entity's sheet, e.g. Vendor in the ledger. Blank cells just skip it instead of rejecting the row.